POLICY-AS-CODE DOCUMENTATION

explico

Readable documentation of Rego policies, true by construction.

A Kotlin library and CLI that drives the real OPA binary

THE PROBLEM

Policy-as-code is hard to read, and harder to trust

Executives and auditors can’t read Rego

The people accountable for a control have no way to check what the code really does.

Hand-written docs drift

The wording and the logic part company the day someone edits a rule.

AI summaries sound right

A paraphrase of logic is a claim nobody verified. Plausible is not the same as true.

WHAT IT IS

Every statement in the output is true by construction.

Derived, not described

Rendered only from the Rego AST and its standard METADATA annotations.

Source when unsure

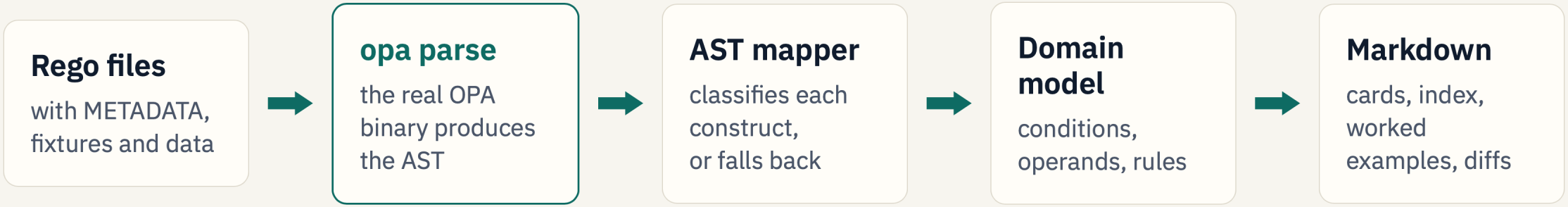
Anything it can’t render exactly appears as clearly marked, verbatim Rego.

Deterministic

The same input directory gives byte-identical output, on every run.

HOW IT WORKS

OPA decides what the policy means. explico maps it.



Rego is never parsed in our code. If OPA accepts a policy, explico can render it. If OPA rejects it, so do we.

INPUT AND OUTPUT

A Rego rule, and what a reader sees

Rego

```
deny contains msg if {
  input.deployment.environment == "production"
  count({a |
    some a in input.change.approvals
    a.role == "release-manager"
  }) == 0
  msg := "no release manager approval is recorded"
}
```

Rendered

All of the following are true:

- deployment ▸ environment is "production"
- (the set of change ▸ approvals ▸ [each a] such that for some a in change ▸ approvals; change ▸ approvals ▸ [each a] ▸ role is "release-manager") is empty

Produces: “no release manager approval is recorded”

THE GUARANTEE IN PRACTICE

What it can’t render, it says so

⚠ not rendered — shown as source:

```
every check in input.pipeline.checks {
  check.status == "passed"
}
```

Rendering coverage: 0 of 1 conditions

Package rendering coverage: 8 of 9 conditions (88%)

Coverage is reported on every card and every package.

A comprehension renders only when every part of it does, never half of one.

A fallback means “read this bit yourself”, not “something is broken”.

WORKED EXAMPLES

Evaluated by OPA, never predicted

From the rendered REL-004 card

unsigned artifact in freeze window — denied

> “deployment falls inside freeze window year-end change freeze” (Situation 2)
>
> “no release manager approval is recorded” (Situation 1)

approved standard release — allowed

hotfix without change ticket — allowed

1 You write small JSON fixtures.

2 opa eval runs each one against the policy at generation time.

3 Each message is matched back to the situation that produced it.

CHANGE REVIEW

A semantic diff, not a text diff

Category	What the report shows
ADDED	The new control’s full card
REMOVED	A removal notice and the old card
LOGIC_CHANGED	The rule’s old and new source as a unified diff
DOCS_CHANGED	A before and after table of title, description and frameworks
UNCHANGED	Nothing at all

Reformatting, comments and variable renames do not register as changes.

Controls are matched by control-id, so renaming a rule keeps its history.

USING IT

A command line, or a library call

Command line

```
explico render policies --out docs \
    --examples examples --data data.json

explico diff old/ new/ --out report.md

explico demo
```

Kotlin or Java

```
val policies = Explico.load(policyDir)
val docs = Explico.render(policies, policyDir)

docs.files // name to Markdown
docs.coverage.percent
```

Needs Java 17 or newer and the opa 1.x binary. Policies in the legacy Rego v0 syntax work with --rego-version v0.

IN YOUR CODE

Load, render, diff: three calls

Kotlin (the same calls work from Java)

```
val policySet = Explico.load(policyDir)
val examples = Explico.loadExamples(examplesDir)
val docs = Explico.render(
    policySet, policyDir, examples, dataFile)

docs.files["index.md"] // Markdown text
docs.coverage.percent // 95

val report = Explico.diff(oldSet, newSet)
report.entries // ADDED, LOGIC_CHANGED, ...
report.markdown // the change report
```

load parses through opa and returns a model you can reuse.

render returns the Markdown in memory. Write it wherever you like.

diff compares two loaded versions. The entries are typed, so a build can act on them.

Get it from Maven Central as io.github.wakaleo:explico. It needs Java 17 or newer.

EVIDENCE

Tested against real-world Rego

97%

of 964 real Rego files (Gatekeeper, Trivy, Regal, conftest) render. The rest are rejected by OPA itself.

0

mapper crashes. A rule that fails to map is isolated and shown as source, never a blank.

95%

of conditions rendered on the reference pack. About 60% on real-world code; the rest is shown as source.

More than 800 automated tests on Linux, macOS and Windows, a frozen acceptance contract, byte-exact golden outputs, and dependencies scanned against the OSV vulnerability database.

HONEST LIMITS

Where it stops today

About 60% on real code

Constructs it won’t paraphrase (every, with, else) and calls nobody can resolve stay as source.

Some examples are withheld

Several same-named boolean rules show no worked examples: a verdict can’t be pinned to one of them.

Early release (0.1.x)

On Maven Central and tested on Linux, macOS and Windows. Pre-1.0, so the API can still change.

You choose the Rego version

Version 1 is the default and legacy v0 is selected explicitly. It is never guessed.

NEXT STEP

See it on a real policy set

```
java -jar explico.jar demo
```

Renders a five-policy release-governance pack with worked examples.

Then read docs/showcase: Rego source beside the rendered output.